

Experiment No. – 13							
Date of Performance:	06-08-2026						
Date of Submission:	07-08-2026						
Conceptual Understanding (02)	Execution (1.5)	Problem Solving & Debugging(02)	Professional Conduct & Lab Ethics(02)	Output/ Result Accuracy (1.5)	Documentation & Result Analysis (01)	Experiment Total (10)	Sign with Date

Experiment No. 13 Microproject

13.1 Aim: To develop a Statistics Toolkit desktop application using Java Swing, AWT Graphics, and SQLite JDBC for statistical analysis, correlation, linear regression, histogram visualization, and database persistence.

13.2 Course Outcome:

- Apply fundamental Java programming concepts, including data types, operators, control structures, and functions, to develop basic programs.
- Develop Java programs using object-oriented programming principles such as classes, objects, constructors, and method overloading.
- Implement inheritance, interfaces, and packages to achieve code reusability and modular program design.
- Develop robust Java applications using exception handling and multithreading techniques.

- Design graphical user interface (GUI) based applications using AWT and Swing components with event-driven programming.
- Develop database-driven Java applications using JDBC and demonstrate basic understanding of modern frameworks such as SpringBoot.

13.3 Learning Objectives:

- Understand implementation of descriptive statistical algorithms.
- Develop GUI-based desktop applications using Java Swing.
- Implement Pearson correlation and linear regression.
- Create custom histogram visualization using Java AWT Graphics.
- Store computed statistical results using SQLite JDBC.

13.4 Requirement:

Software:

- Java JDK 17 or later
- SQLite JDBC Driver
- Any Java IDE (IntelliJ IDEA/Eclipse/NetBeans/VS Code)
- SQLite Database (statistics.db)

Hardware:

- Windows/Linux/macOS
- Minimum 4 GB RAM
- 200 MB free storage

13.5 Related Theory:

All details regarding mini project

StatisticsToolkit.java is a self-contained Java desktop application built using Java Swing, AWT Graphics, and SQLite JDBC.

Architecture:

StatisticsToolkit (main)

- > Initializes Database & Starts GUI
 - > MainFrame
 - > Home
 - > Statistics
 - > Correlation
 - > Regression
 - > Histogram
 - > About

Main Components:

1. StatisticsToolkit:

Initializes the SQLite database and launches the GUI using `SwingUtilities.invokeLater()`.

2. `MainFrame`:

Creates an 800×600 window with a `JTabbedPane` containing six modules.

3. `StatisticsCalculator`:

Implements Mean, Median, Mode, Variance, Standard Deviation, Min, Max, Count, and Quartiles without external libraries.

4. `StatisticsPanel`:

Accepts comma-separated numerical input, computes statistics, displays results, and stores them in the database.

5. `CorrelationCalculator`:

Computes the Pearson Correlation Coefficient and classifies the relationship as Strong Positive, Positive, Weak, Negative, or Strong Negative.

6. `CorrelationPanel`:

Accepts paired X and Y values, validates input, and displays the correlation coefficient.

7. `RegressionCalculator`:

Calculates slope, intercept, and predicts values using the equation:

$$Y = b + mX$$

8. `RegressionPanel`:

Displays the regression equation and predicts output values for user-specified X.

9. `HistogramDrawer`:

Uses Java Graphics by overriding `paintComponent(Graphics g)` to draw scalable histogram bars, axes, labels, and frequency bins.

10. `HistogramPanel`:

Parses numeric input and updates the histogram dynamically.

11. `AboutPanel`:

Displays project information and developer notes.

12. `DatabaseConnector`:

Creates `statistics.db` and stores dataset name, mean, median, mode, variance, and standard deviation using `PreparedStatement`.

13.6 Procedure:

1. Install Java JDK and SQLite JDBC.
2. Compile and run `StatisticsToolkit.java`.
3. Database initializes automatically.
4. Enter dataset values in the Statistics tab.
5. Compute descriptive statistics and save results.
6. Perform correlation analysis using paired datasets.

7. Generate regression equation and predictions.
8. Draw histogram from input data.
9. View project information in the About tab.

13.7 Program and Output:

```
// =====
// File: Main.java
// =====

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.*;

/**
 * Main entry point for the Statistics & Data Analysis Toolkit.
 */
public class StatisticsToolkit {
    public static void main(String[] args) {
        try {
            DatabaseConnector.initializeDatabase();
        } catch (ClassNotFoundException e) {
            JOptionPane.showMessageDialog(null, "SQLite JDBC driver not found. Add sqlite-jdbc.jar to
the classpath.", "Database Error", JOptionPane.ERROR_MESSAGE);
            return;
        } catch (java.sql.SQLException e) {
            JOptionPane.showMessageDialog(null, "Unable to initialize database:\n" + e.getMessage(),
"Database Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
    }

    // Run GUI construction on the Event Dispatch Thread (EDT)
    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
            MainFrame frame = new MainFrame();
            frame.setVisible(true);
        }
    });
}

// =====
// File: MainFrame.java
```

```
// =====

/**
 * Main window container for the application.
 * Manages the tabs for different features.
 */
class MainFrame extends JFrame {
    public MainFrame() {
        setTitle("Statistics & Data Analysis Toolkit");
        setSize(800, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null); // Center on screen

        JTabbedPane tabbedPane = new JTabbedPane();

        tabbedPane.addTab("Home", createHomePanel());
        tabbedPane.addTab("Statistics", new StatisticsPanel());
        tabbedPane.addTab("Correlation", new CorrelationPanel());
        tabbedPane.addTab("Regression", new RegressionPanel());
        tabbedPane.addTab("Histogram", new HistogramPanel());
        tabbedPane.addTab("About", new AboutPanel());

        add(tabbedPane);
    }

    private JPanel createHomePanel() {
        JPanel panel = new JPanel();
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));

        JLabel welcomeLabel = new JLabel("Welcome to Statistics Toolkit");
        welcomeLabel.setAlignmentX(JPanel.CENTER_ALIGNMENT);
        welcomeLabel.setFont(new java.awt.Font("Arial", java.awt.Font.BOLD, 28));

        JLabel subLabel = new JLabel("Select a tab above to begin analysis");
        subLabel.setAlignmentX(JPanel.CENTER_ALIGNMENT);
        subLabel.setFont(new java.awt.Font("Arial", java.awt.Font.PLAIN, 16));

        panel.add(Box.createVerticalStrut(200));
        panel.add(welcomeLabel);
        panel.add(Box.createVerticalStrut(10));
        panel.add(subLabel);
        return panel;
    }
}

// =====
// File: StatisticsCalculator.java
// =====
```

```

/**
 * Contains core logic for calculating statistical values.
 */
class StatisticsCalculator {

    public static double getMean(double[] data) {
        double sum = 0;
        for (double num : data) sum += num;
        return sum / data.length;
    }

    public static double getMedian(double[] data) {
        double[] sorted = data.clone();
        Arrays.sort(sorted);
        int n = sorted.length;
        if (n % 2 == 0) {
            return (sorted[n / 2 - 1] + sorted[n / 2]) / 2.0;
        } else {
            return sorted[n / 2];
        }
    }

    public static double getMode(double[] data) {
        HashMap<Double, Integer> frequencyMap = new HashMap<>();
        for (double num : data) {
            frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
        }
        double mode = data[0];
        int maxCount = 0;
        for (Map.Entry<Double, Integer> entry : frequencyMap.entrySet()) {
            if (entry.getValue() > maxCount) {
                maxCount = entry.getValue();
                mode = entry.getKey();
            }
        }
        return mode;
    }

    public static double getVariance(double[] data) {
        double mean = getMean(data);
        double sumSq = 0;
        for (double num : data) {
            sumSq += Math.pow(num - mean, 2);
        }
        return sumSq / data.length; // Population variance
    }
}

```

```

public static double getStandardDeviation(double[] data) {
    return Math.sqrt(getVariance(data));
}

public static double getMin(double[] data) {
    double min = data[0];
    for (double num : data) if (num < min) min = num;
    return min;
}

public static double getMax(double[] data) {
    double max = data[0];
    for (double num : data) if (num > max) max = num;
    return max;
}

public static int getCount(double[] data) {
    return data.length;
}

public static double getQ1(double[] data) {
    double[] sorted = data.clone();
    Arrays.sort(sorted);
    int n = sorted.length;
    return getMedian(Arrays.copyOfRange(sorted, 0, n / 2));
}

public static double getQ2(double[] data) {
    return getMedian(data);
}

public static double getQ3(double[] data) {
    double[] sorted = data.clone();
    Arrays.sort(sorted);
    int n = sorted.length;
    if (n % 2 == 0) {
        return getMedian(Arrays.copyOfRange(sorted, n / 2, n));
    } else {
        return getMedian(Arrays.copyOfRange(sorted, n / 2 + 1, n));
    }
}

// =====
// File: StatisticsPanel.java
// =====

/**

```

```

* GUI Panel for Statistics calculations.
*/
class StatisticsPanel extends JPanel {
    private JTextField nameField, dataField;
    private JTextArea resultArea;

    public StatisticsPanel() {
        setLayout(new BorderLayout(10, 10));
        setBorder(BorderFactory.createEmptyBorder(15, 15, 15, 15));

        // Input Panel
        JPanel inputPanel = new JPanel(new GridLayout(2, 2, 10, 10));
        inputPanel.add(new JLabel("Dataset Name:"));
        nameField = new JTextField();
        inputPanel.add(nameField);

        inputPanel.add(new JLabel("Numbers (comma-separated):"));
        dataField = new JTextField();
        inputPanel.add(dataField);

        // Buttons Panel
        JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 10));
        JButton analyzeButton = new JButton("Analyze");
        JButton resetButton = new JButton("Reset");
        JButton exitButton = new JButton("Exit");
        buttonPanel.add(analyzeButton);
        buttonPanel.add(resetButton);
        buttonPanel.add(exitButton);

        // Result Area
        resultArea = new JTextArea();
        resultArea.setEditable(false);
        resultArea.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane = new JScrollPane(resultArea);

        // Assemble Top
        JPanel topPanel = new JPanel(new BorderLayout());
        topPanel.add(inputPanel, BorderLayout.CENTER);
        topPanel.add(buttonPanel, BorderLayout.SOUTH);

        add(topPanel, BorderLayout.NORTH);
        add(scrollPane, BorderLayout.CENTER);

        // Actions
        analyzeButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                analyzeData();
            }
        });
    }
}

```

```

    }
    });

    resetButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            nameField.setText("");
            dataField.setText("");
            resultArea.setText("");
        }
    });

    exitButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            System.exit(0);
        }
    });
}

private void analyzeData() {
    try {
        String input = dataField.getText().trim();
        if (input.isEmpty()) {
            throw new IllegalArgumentException("Data cannot be empty.");
        }
        String[] parts = input.split(",");
        double[] data = new double[parts.length];
        for (int i = 0; i < parts.length; i++) {
            data[i] = Double.parseDouble(parts[i].trim());
        }

        StringBuilder sb = new StringBuilder();
        sb.append("Dataset: ").append(nameField.getText()).append("\n");
        sb.append("-----\n");
        sb.append(String.format("Mean:           %.4f\n", StatisticsCalculator.getMean(data)));
        sb.append(String.format("Median:        %.4f\n", StatisticsCalculator.getMedian(data)));
        sb.append(String.format("Mode:          %.4f\n", StatisticsCalculator.getMode(data)));
        sb.append(String.format("Variance:      %.4f\n", StatisticsCalculator.getVariance(data)));
        sb.append(String.format("Standard Deviation: %.4f\n",
StatisticsCalculator.getStandardDeviation(data)));
        sb.append(String.format("Minimum:       %.4f\n", StatisticsCalculator.getMin(data)));
        sb.append(String.format("Maximum:       %.4f\n", StatisticsCalculator.getMax(data)));
        sb.append(String.format("Count:         %d\n", StatisticsCalculator.getCount(data)));
        sb.append(String.format("Quartile 1:    %.4f\n", StatisticsCalculator.getQ1(data)));
        sb.append(String.format("Quartile 2:    %.4f\n", StatisticsCalculator.getQ2(data)));
        sb.append(String.format("Quartile 3:    %.4f\n", StatisticsCalculator.getQ3(data)));
    }
}

```

```

        resultArea.setText(sb.toString());

        try {
            DatabaseConnector.insertStatistics(
                nameField.getText().trim().isEmpty() ? "Unnamed Dataset" :
nameField.getText().trim(),
                StatisticsCalculator.getMean(data),
                StatisticsCalculator.getMedian(data),
                StatisticsCalculator.getMode(data),
                StatisticsCalculator.getVariance(data),
                StatisticsCalculator.getStandardDeviation(data)
            );
        } catch (ClassNotFoundException e) {
            JOptionPane.showMessageDialog(this, "SQLite JDBC driver not found. Add sqlite-jdbc.jar
to the classpath.", "Database Error", JOptionPane.ERROR_MESSAGE);
        } catch (java.sql.SQLException e) {
            JOptionPane.showMessageDialog(this, "Unable to save results:\n" + e.getMessage(),
"Database Error", JOptionPane.ERROR_MESSAGE);
        }

        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Invalid numbers. Please use only numbers and
commas.", "Input Error", JOptionPane.ERROR_MESSAGE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
        }
    }
}

// =====
// File: CorrelationCalculator.java
// =====

/**
 * Core logic for Pearson Correlation calculations.
 */
class CorrelationCalculator {
    public static double getPearsonCorrelation(double[] x, double[] y) throws
IllegalArgumentException {
        if (x.length != y.length) {
            throw new IllegalArgumentException("X and Y must have the same number of elements.");
        }
        int n = x.length;
        double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0, sumY2 = 0;

        for (int i = 0; i < n; i++) {
            sumX += x[i];

```

```

        sumY += y[i];
        sumXY += x[i] * y[i];
        sumX2 += x[i] * x[i];
        sumY2 += y[i] * y[i];
    }

    double numerator = (n * sumXY) - (sumX * sumY);
    double denominator = Math.sqrt((n * sumX2 - sumX * sumX) * (n * sumY2 - sumY * sumY));

    if (denominator == 0) return 0;
    return numerator / denominator;
}

public static String getRelationship(double r) {
    if (r >= 0.7) return "Strong Positive";
    if (r > 0.3) return "Positive";
    if (r >= -0.3) return "Weak";
    if (r > -0.7) return "Negative";
    return "Strong Negative";
}
}

// =====
// File: CorrelationPanel.java
// =====

/**
 * GUI Panel for Correlation calculations.
 */
class CorrelationPanel extends JPanel {
    private JTextField xField, yField;
    private JTextArea resultArea;

    public CorrelationPanel() {
        setLayout(new BorderLayout(10, 10));
        setBorder(BorderFactory.createEmptyBorder(15, 15, 15, 15));

        JPanel inputPanel = new JPanel(new GridLayout(3, 2, 10, 10));
        inputPanel.add(new JLabel("X Values (comma-separated):"));
        xField = new JTextField();
        inputPanel.add(xField);

        inputPanel.add(new JLabel("Y Values (comma-separated):"));
        yField = new JTextField();
        inputPanel.add(yField);

        JButton calcButton = new JButton("Calculate Correlation");
        inputPanel.add(new JLabel()); // Spacer

```

```

inputPanel.add(calcButton);

resultArea = new JTextArea();
resultArea.setEditable(false);
resultArea.setFont(new Font("Monospaced", Font.BOLD, 14));
JScrollPane scrollPane = new JScrollPane(resultArea);

add(inputPanel, BorderLayout.NORTH);
add(scrollPane, BorderLayout.CENTER);

calcButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        calculateCorrelation();
    }
});
}

private void calculateCorrelation() {
    try {
        String[] xParts = xField.getText().split(",");
        String[] yParts = yField.getText().split(",");
        if (xParts.length != yParts.length) {
            throw new IllegalArgumentException("X and Y datasets must have the same number of
elements.");
        }

        double[] x = new double[xParts.length];
        double[] y = new double[yParts.length];

        for (int i = 0; i < xParts.length; i++) {
            x[i] = Double.parseDouble(xParts[i].trim());
            y[i] = Double.parseDouble(yParts[i].trim());
        }

        double r = CorrelationCalculator.getPearsonCorrelation(x, y);
        String relationship = CorrelationCalculator.getRelationship(r);

        String output = String.format("Correlation Coefficient (r): %.4f\n\nRelationship: %s", r,
relationship);
        resultArea.setText(output);

    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Please enter valid numbers.", "Input Error",
JOptionPane.ERROR_MESSAGE);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
    }
}

```

```

    }
}

// =====
// File: RegressionCalculator.java
// =====

/**
 * Core logic for Linear Regression ( $y = mx + b$ ).
 */
class RegressionCalculator {

    public static double getSlope(double[] x, double[] y) throws IllegalArgumentException {
        if (x.length != y.length) {
            throw new IllegalArgumentException("X and Y must have the same number of elements.");
        }
        int n = x.length;
        double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0;

        for (int i = 0; i < n; i++) {
            sumX += x[i];
            sumY += y[i];
            sumXY += x[i] * y[i];
            sumX2 += x[i] * x[i];
        }

        double denominator = (n * sumX2) - (sumX * sumX);
        if (denominator == 0) return 0;

        return ((n * sumXY) - (sumX * sumY)) / denominator;
    }

    public static double getIntercept(double[] x, double[] y, double slope) {
        double sumX = 0, sumY = 0;
        for (int i = 0; i < x.length; i++) {
            sumX += x[i];
            sumY += y[i];
        }
        return (sumY - slope * sumX) / x.length;
    }

    public static double predict(double slope, double intercept, double xValue) {
        return intercept + (slope * xValue);
    }
}

// =====

```

```

// File: RegressionPanel.java
// =====

/**
 * GUI Panel for Linear Regression calculations.
 */
class RegressionPanel extends JPanel {
    private JTextField xField, yField, predictField;
    private JTextArea resultArea;

    public RegressionPanel() {
        setLayout(new BorderLayout(10, 10));
        setBorder(BorderFactory.createEmptyBorder(15, 15, 15, 15));

        JPanel inputPanel = new JPanel(new GridLayout(4, 2, 10, 10));
        inputPanel.add(new JLabel("X Values (comma-separated):"));
        xField = new JTextField();
        inputPanel.add(xField);

        inputPanel.add(new JLabel("Y Values (comma-separated):"));
        yField = new JTextField();
        inputPanel.add(yField);

        inputPanel.add(new JLabel("Predict X:"));
        predictField = new JTextField();
        inputPanel.add(predictField);

        JButton calcButton = new JButton("Calculate Regression");
        inputPanel.add(new JLabel()); // Spacer
        inputPanel.add(calcButton);

        resultArea = new JTextArea();
        resultArea.setEditable(false);
        resultArea.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane = new JScrollPane(resultArea);

        add(inputPanel, BorderLayout.NORTH);
        add(scrollPane, BorderLayout.CENTER);

        calcButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                calculateRegression();
            }
        });
    }

    private void calculateRegression() {

```

```

try {
    String[] xParts = xField.getText().split(",");
    String[] yParts = yField.getText().split(",");

    if (xParts.length != yParts.length) {
        throw new IllegalArgumentException("X and Y must have the same length.");
    }

    double[] x = new double[xParts.length];
    double[] y = new double[yParts.length];
    for (int i = 0; i < xParts.length; i++) {
        x[i] = Double.parseDouble(xParts[i].trim());
        y[i] = Double.parseDouble(yParts[i].trim());
    }

    double slope = RegressionCalculator.getSlope(x, y);
    double intercept = RegressionCalculator.getIntercept(x, y, slope);

    StringBuilder sb = new StringBuilder();
    sb.append("-----\n");
    sb.append(String.format("Slope (m):   %.4f\n", slope));
    sb.append(String.format("Intercept (b): %.4f\n", intercept));
    sb.append(String.format("Regression Equation: Y = %.4f + %.4fX\n", intercept, slope));
    sb.append("-----\n");

    String predictText = predictField.getText().trim();
    if (!predictText.isEmpty()) {
        double predictX = Double.parseDouble(predictText);
        double predictedY = RegressionCalculator.predict(slope, intercept, predictX);
        sb.append(String.format("\nPredicted Value for X = %.2f:\n", predictX));
        sb.append(String.format("Y = %.4f", predictedY));
    }

    resultArea.setText(sb.toString());

} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this, "Please enter valid numbers.", "Input Error",
JOptionPane.ERROR_MESSAGE);
} catch (Exception ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
}
}
}

// =====
// File: HistogramDrawer.java
// =====

```

```

/**
 * Custom JPanel to draw a histogram using Java Swing Graphics.
 */
class HistogramDrawer extends JPanel {
    private double[] data;

    public void setData(double[] data) {
        this.data = data;
        repaint(); // Request a redraw with the new data
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);

        if (data == null || data.length == 0) {
            g.setFont(new Font("Arial", Font.ITALIC, 14));
            g.drawString("Enter data and click Draw to see the histogram.", 20, 30);
            return;
        }

        // Find min and max
        double min = data[0], max = data[0];
        for (double v : data) {
            if (v < min) min = v;
            if (v > max) max = v;
        }

        // Setup bins
        int numBins = 10;
        int[] bins = new int[numBins];
        double binWidth = (max - min) / numBins;
        if (binWidth == 0) binWidth = 1; // Prevent division by zero if all values are same

        for (double v : data) {
            int binIndex = (int) ((v - min) / binWidth);
            if (binIndex >= numBins) binIndex = numBins - 1; // Put max value in the last bin
            bins[binIndex]++;
        }

        // Find max frequency for scaling
        int maxFreq = 0;
        for (int freq : bins) {
            if (freq > maxFreq) maxFreq = freq;
        }

        // Drawing dimensions

```

```

int width = getWidth();
int height = getHeight();
int padding = 50;

int barWidth = (width - 2 * padding) / numBins;

// Draw Axes
g.setColor(Color.BLACK);
g.drawLine(padding, height - padding, width - padding, height - padding); // X-axis
g.drawLine(padding, padding, padding, height - padding); // Y-axis

// Draw Bars
g.setColor(new Color(70, 130, 180)); // Steel Blue
for (int i = 0; i < numBins; i++) {
    // Calculate height of bar relative to max frequency
    int barHeight = (int) (((double) bins[i] / (double) maxFreq) * (height - 2 * padding));
    int x = padding + i * barWidth;
    int y = height - padding - barHeight;

    // Fill bar
    g.fillRect(x, y, barWidth - 2, barHeight);

    // Draw border
    g.setColor(Color.BLACK);
    g.drawRect(x, y, barWidth - 2, barHeight);
    g.setColor(new Color(70, 130, 180)); // Reset color
}

// Draw Labels
g.setColor(Color.BLACK);
g.setFont(new Font("Arial", Font.PLAIN, 12));
g.drawString("0", padding - 15, height - padding);
g.drawString(String.valueOf(maxFreq), padding - 25, padding + 10);
}
}

// =====
// File: HistogramPanel.java
// =====

/**
 * GUI Panel for the Histogram feature.
 */
class HistogramPanel extends JPanel {
    private JTextField dataField;
    private HistogramDrawer histogramDrawer;

    public HistogramPanel() {

```

```

setLayout(new BorderLayout(10, 10));
setBorder(BorderFactory.createEmptyBorder(15, 15, 15, 15));

JPanel inputPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 10));
inputPanel.add(new JLabel("Data (comma-separated):"));
dataField = new JTextField(35);
inputPanel.add(dataField);

JButton drawButton = new JButton("Draw Histogram");
inputPanel.add(drawButton);

histogramDrawer = new HistogramDrawer();
histogramDrawer.setBackground(Color.WHITE);
histogramDrawer.setBorder(BorderFactory.createLineBorder(Color.LIGHT_GRAY));

add(inputPanel, BorderLayout.NORTH);
add(histogramDrawer, BorderLayout.CENTER);

drawButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        try {
            String input = dataField.getText().trim();
            if (input.isEmpty()) {
                throw new IllegalArgumentException("Please enter data.");
            }
            String[] parts = input.split(",");
            double[] data = new double[parts.length];
            for (int i = 0; i < parts.length; i++) {
                data[i] = Double.parseDouble(parts[i].trim());
            }
            histogramDrawer.setData(data);
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(HistogramPanel.this, "Invalid data. Please enter
numbers separated by commas.", "Input Error", JOptionPane.ERROR_MESSAGE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(HistogramPanel.this, ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
        }
    }
});
}
}

// =====
// File: AboutPanel.java
// =====

```

```

/**
 * Panel to display project information and features.
 */
class AboutPanel extends JPanel {
    public AboutPanel() {
        setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));
        setBorder(BorderFactory.createEmptyBorder(30, 40, 30, 40));

        JLabel titleLabel = new JLabel("Statistics & Data Analysis Toolkit");
        titleLabel.setFont(new Font("Arial", Font.BOLD, 26));
        titleLabel.setAlignmentX(Component.CENTER_ALIGNMENT);

        JLabel devLabel = new JLabel("Developer: Java Student");
        devLabel.setFont(new Font("Arial", Font.ITALIC, 16));
        devLabel.setAlignmentX(Component.CENTER_ALIGNMENT);

        JTextArea descArea = new JTextArea(
            "Description:\n" +
            "A beginner-friendly desktop application built entirely with Core Java and Swing " +
            "for analyzing data sets. This project uses no external libraries, implementing " +
            "all mathematical formulas manually.\n\n" +
            "Features:\n" +
            "• Statistics Calculator (Mean, Median, Mode, Variance, SD, Quartiles, etc.)\n" +
            "• Correlation Calculator (Pearson Correlation Coefficient)\n" +
            "• Linear Regression (Predict values based on linear trends)\n" +
            "• Histogram Drawer (Custom AWT graphics visualization)\n" +
            "• Modular Object-Oriented Architecture\n" +
            "• Clean and Simple UI"
        );
        descArea.setEditable(false);
        descArea.setOpaque(false);
        descArea.setFont(new Font("Arial", Font.PLAIN, 15));
        descArea.setLineWrap(true);
        descArea.setWrapStyleWord(true);
        descArea.setMargin(new Insets(20, 20, 20, 20));

        add(titleLabel);
        add(Box.createRigidArea(new Dimension(0, 10)));
        add(devLabel);
        add(Box.createRigidArea(new Dimension(0, 30)));
        add(descArea);
    }
}

// =====
// File: DatabaseConnector.java
// =====

```

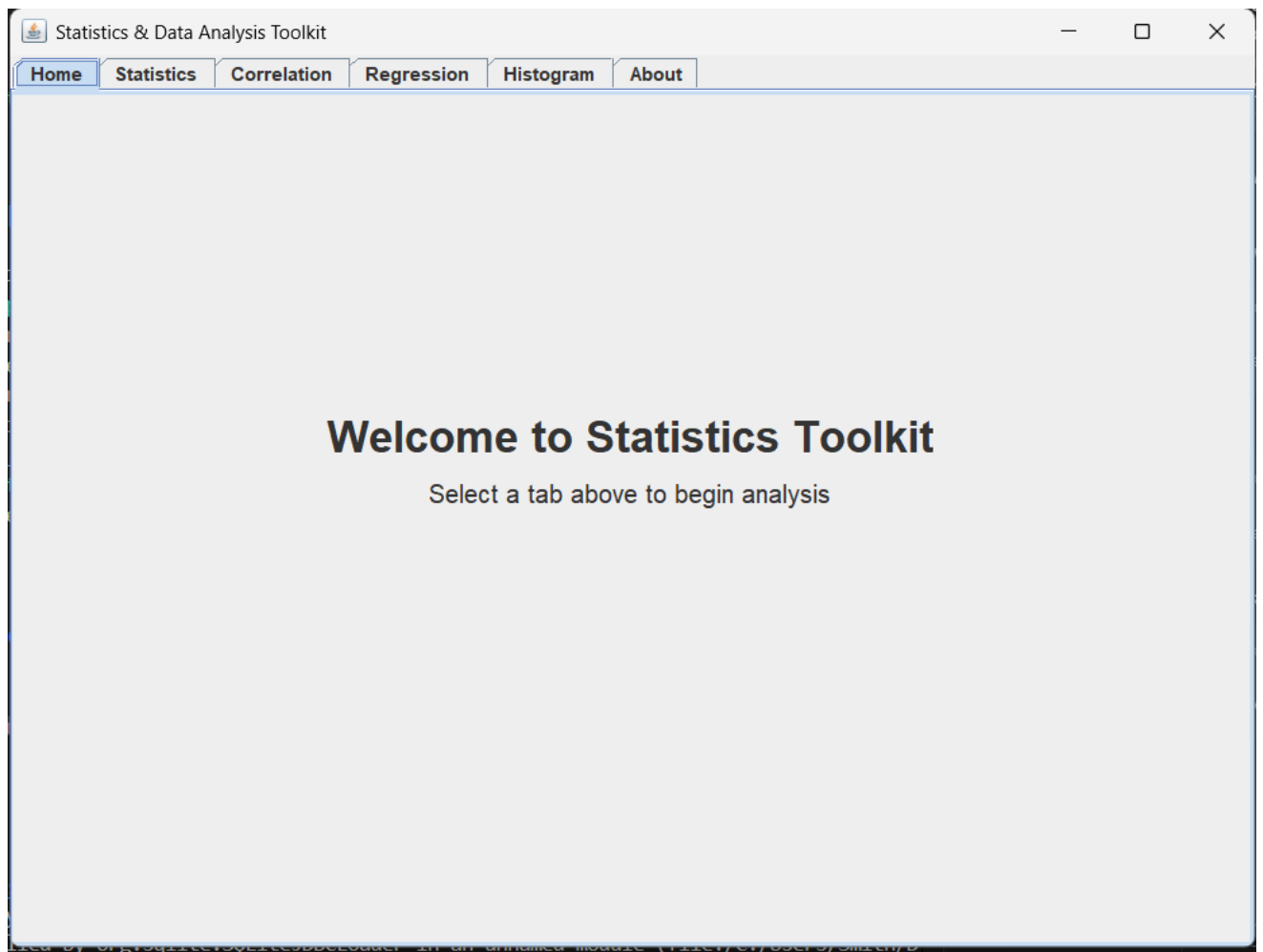
```

/**
 * Handles SQLite database initialization and recording statistics results.
 */
class DatabaseConnector {
    private static final String DB_URL = "jdbc:sqlite:statistics.db";

    public static void initializeDatabase() throws ClassNotFoundException, java.sql.SQLException {
        Class.forName("org.sqlite.JDBC");
        try (java.sql.Connection conn = java.sql.DriverManager.getConnection(DB_URL);
            java.sql.Statement stmt = conn.createStatement()) {
            String sql = "CREATE TABLE IF NOT EXISTS statistics (" +
                "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                "dataset_name TEXT, " +
                "mean REAL, " +
                "median REAL, " +
                "mode REAL, " +
                "variance REAL, " +
                "std_dev REAL" +
                ");";
            stmt.execute(sql);
        }
    }

    public static void insertStatistics(String datasetName, double mean, double median, double mode,
        double variance, double stdDev) throws ClassNotFoundException, java.sql.SQLException {
        Class.forName("org.sqlite.JDBC");
        String sql = "INSERT INTO statistics(dataset_name, mean, median, mode, variance, std_dev)
VALUES(?, ?, ?, ?, ?, ?)";
        try (java.sql.Connection conn = java.sql.DriverManager.getConnection(DB_URL);
            java.sql.PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, datasetName);
            pstmt.setDouble(2, mean);
            pstmt.setDouble(3, median);
            pstmt.setDouble(4, mode);
            pstmt.setDouble(5, variance);
            pstmt.setDouble(6, stdDev);
            pstmt.executeUpdate();
        }
    }
}

```



Statistics & Data Analysis Toolkit

Home Statistics Correlation Regression Histogram About

Dataset Name:

Numbers (comma-separated):

Analyze Reset Exit

Dataset: Statistics

Mean:	74.1000
Median:	79.0000
Mode:	67.0000
Variance:	299.2900
Standard Deviation:	17.3000
Minimum:	45.0000
Maximum:	92.0000
Count:	10
Quartile 1:	67.0000
Quartile 2:	79.0000
Quartile 3:	89.0000

Statistics & Data Analysis Toolkit

Home Statistics **Correlation** Regression Histogram About

X Values (comma-separated):

Y Values (comma-separated):

Correlation Coefficient (r): 1.0000

Relationship: Strong Positive

Statistics & Data Analysis Toolkit

Home Statistics Correlation **Regression** Histogram About

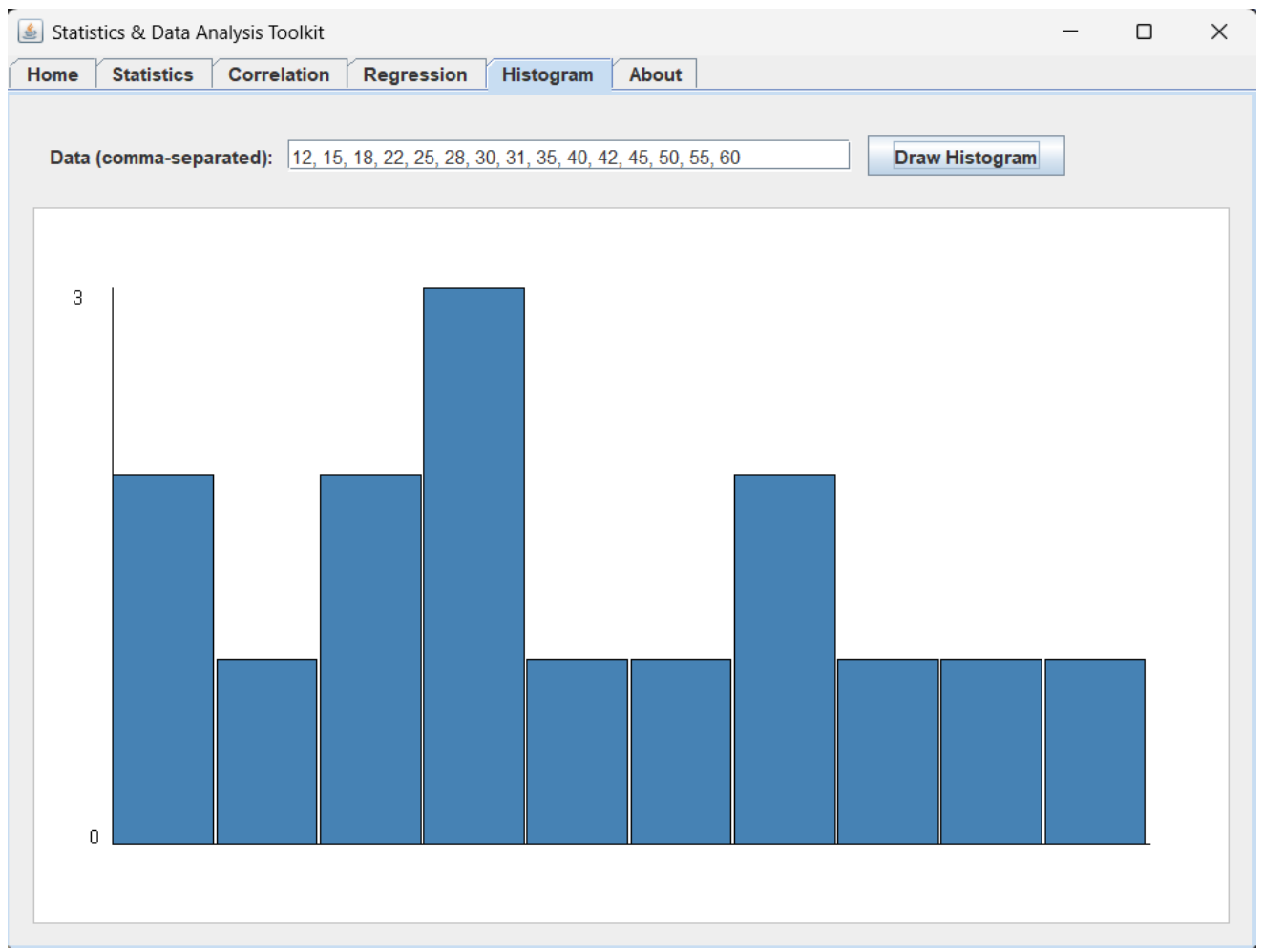
X Values (comma-separated):

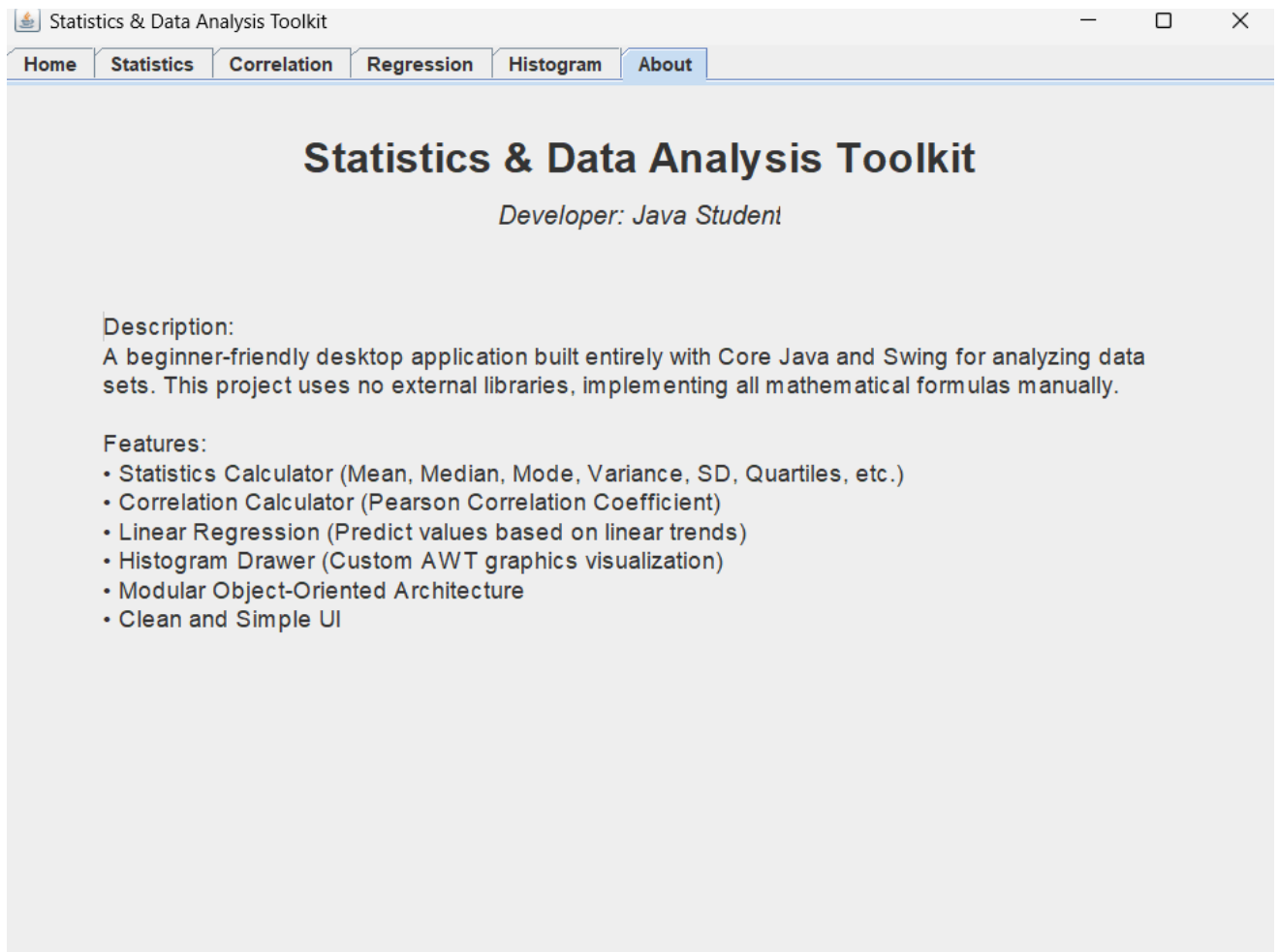
Y Values (comma-separated):

Predict X:

Slope (m): 2.0000
Intercept (b): 1.0000
Regression Equation: $Y = 1.0000 + 2.0000X$

Predicted Value for X = 10.00:
Y = 21.0000





13.8 Conclusion:

The Statistics Toolkit successfully integrates Java Swing, AWT Graphics, and SQLite JDBC into a single desktop application. It demonstrates object-oriented programming, GUI development, database connectivity, statistical computation, regression analysis, correlation analysis, and custom graphics rendering, making it an effective Java microproject.